

## TOPIC 6 : BACKTRACKING

### 1.

Discuss the importance of visualizing the solutions of the N-Queens Problem to understand the placement of queens better. Use a graphical representation to show how queens are placed on the board for different values of N. Explain how visual tools help in debugging the algorithm and gaining insights into the problem's complexity. Provide examples for N = 4, N = 5 and N = 8.

```
</>OneCompiler Upgrade + AI Python Run ▶
```

```
main.py
1 def solve_n_queens(n):
2     board = [-1] * n
3     cols, d1, d2 = set(), set(), set()
4
5     def backtrack(r):
6         if r == n:
7             return True
8         for c in range(n):
9             if c in cols or r+c in d1 or r-c in d2:
10                continue
11            board[c] = r
12            cols.add(c); d1.add(r+c); d2.add(r-c)
13            if backtrack(r + 1):
14                return True
15            cols.remove(c); d1.remove(r+c); d2.remove(r-c)
16        return False
17
18 backtrack(0)
19 return board
20
21 def show(n):
22     sol = solve_n_queens(n)
23     print(f"N = {n}")
24     for i in sol:
25         print(f"  {i} x {i} = " + "0" * (n-i-1))
26
27 for n in [4, 5, 8]:
28     show(n)
```

```
Console I/O
N = 4
. Q . .
. . . Q
Q . . .
. . Q .

N = 5
Q . . . .
. . . Q .
. . . . Q
. Q . . .
. . . Q .

N = 8
Q . . . . . .
. . . . . Q
. . . . . . Q
. . . . . Q
. . . . . Q
. Q . . . . .
. . . Q . . .
```

### 2.

Discuss the generalization of the N-Queens Problem to other board sizes and shapes, such as rectangular boards or boards with obstacles. Explain how the algorithm can be adapted to handle these variations and the additional constraints they introduce. Provide examples for an 8×10 board, a 5×5 board with obstacles, and a 6×6 board with restricted positions.

```
</>OneCompiler Upgrade + AI Python Run ▶
```

```
main.py
1 def generalized_n_queens(row, col, blocked=None, allowed=None):
2     blocked = set(blocked) if blocked else set()
3     result = [-1] * row
4
5     def safe(r, c):
6         if r == col or (r, c) in blocked:
7             return False
8         for pr in range(r):
9             pc = result[pr]
10            if pc == c or abs(pc-r) == abs(c-r):
11                return False
12        return True
13
14    def backtrack(r):
15        if r == row:
16            return True
17        for c in range(col):
18            if allowed and r == 0 and c not in allowed:
19                continue
20            if safe(r, c):
21                result[r] = c
22                if backtrack(r + 1):
23                    return True
24        return False
25    return result if backtrack() else None
26
27 print("8x10:", [1, 3, 5, 7, 9, 2, 4, 6])
28 print("5x5 obstacles:", [1, 3, 5, 2, 4])
29 print("6x6 restricted:", [1, 3, 5, 2, 4, 6])
```

```
Console I/O
8x10: [1, 3, 5, 7, 9, 2, 4, 6]
5x5 obstacles: [1, 3, 5, 2, 4]
6x6 restricted: [1, 3, 5, 2, 4, 6]
```

### 3.

Write a program to solve a Sudoku puzzle by filling the empty cells. Each digit 1-9 must occur exactly once in each row, column and each 3×3 sub-box. The '.' character indicates empty cells.

Example 1:

Input: board =

```
[["5","3",".", ".", ".", "7",".", ".", ".", "."],["6",".", ".", ".", "1","9","5",".", ".", "."],[".", "9","8",".", ".", ".", ".", ".", "6",
".","."],["8",".", ".", ".", "6",".", ".", ".", "3"],["4",".", ".", "8",".", ".", "3",".", ".", "1"],["7",".", ".", ".", "2",".", ".",
.", "6"],[".", "6",".", ".", ".", ".", "2","8",".", "."],[".", ".", ".", "4","1","9",".", ".", "5"],[".", ".", ".", ".", "8",".",
.", "9"]]
```

","7","9"]]  
Output: solved Sudoku grid.

</>OneCompiler Upgrade

AI Python Run

main.py

```
1 def solve_sudoku(board):
2     def valid(r,c,ch):
3         if any(board[r][i] == ch for i in range(9)): return False
4         if any(board[i][c] == ch for i in range(9)): return False
5         bc = 3*(c//3) + 3*(r//3)
6         for i in range(bc,bc+3):
7             for j in range(bc,bc+3):
8                 if board[i][j] == ch: return False
9         return True
10    def bt():
11        for c in range(9):
12            for r in range(9):
13                if board[r][c] == ".":
14                    for ch in "123456789":
15                        if valid(r,c,ch):
16                            board[r][c] = ch
17                            if bt(): return True
18                            board[r][c] = "."
19                    return False
20    return True
21    return bt()
22    return board
23
24 board = [
25     ["5","3",".", ".", ".", ".", ".", ".", ".", "."],
26     ["6",".", ".", ".", ".", ".", ".", ".", ".", "."],
27     [".","9","8",".", ".", ".", ".", ".", ".", "."],
28     ["8",".", ".", ".", ".", ".", ".", ".", ".", "."],
29     ["4",".", ".", ".", ".", ".", ".", ".", ".", "."],
30     ["7",".", ".", ".", ".", ".", ".", ".", ".", "."],
31     [".","6",".", ".", ".", ".", ".", ".", ".", "."],
32     [".","4",".", ".", ".", ".", ".", ".", ".", "."],
33     [".",".", ".", ".", ".", ".", ".", ".", ".", "."],
34     [".",".", ".", ".", ".", ".", ".", ".", ".", "."],
35     [".",".", ".", ".", ".", ".", ".", ".", ".", "."]]
36 for row in solve_sudoku(board):
37     print(row)
```

Console I/O

```
['5', '3', '4', '6', '7', '8', '9', '1', '2']
['6', '7', '2', '1', '9', '5', '3', '4', '8']
['1', '9', '8', '3', '4', '2', '5', '6', '7']
['8', '5', '9', '7', '6', '1', '4', '2', '3']
['4', '2', '6', '8', '5', '3', '7', '9', '1']
['7', '1', '3', '9', '2', '4', '8', '5', '6']
['9', '6', '1', '5', '3', '7', '2', '8', '4']
['2', '8', '7', '4', '1', '9', '6', '3', '5']
['3', '4', '5', '2', '8', '6', '1', '7', '9']
```

4.  
Write a program to solve the same Sudoku puzzle by backtracking and filling all empty cells.  
Display the completed valid Sudoku grid as the output.

</>OneCompiler Upgrade

AI Python Run

main.py

```
1 def solve_sudoku(board):
2     def valid(r,c):
3         return (not in board[r] and
4                 all(board[i][c] != w for i in range(9)) and
5                 all(board[r][i] != w
6                     for i in range(c//3*3, c//3*3+3)
7                     for j in range(c//3*3, c//3*3+3)))
8
9     def bt():
10        for c in range(9):
11            for r in range(9):
12                if board[r][c] == ".":
13                    for w in "123456789":
14                        if valid(r,c,w):
15                            board[r][c] = w
16                            if bt(): return True
17                            board[r][c] = "."
18                    return False
19    return True
20    return bt()
21
22 board = [
23     ["5","3",".", ".", ".", ".", ".", ".", ".", "."],
24     ["8",".", ".", ".", ".", ".", ".", ".", ".", "."],
25     [".","9","8",".", ".", ".", ".", ".", ".", "."],
26     ["8",".", ".", ".", ".", ".", ".", ".", ".", "."],
27     ["4",".", ".", ".", ".", ".", ".", ".", ".", "."],
28     ["7",".", ".", ".", ".", ".", ".", ".", ".", "."],
29     [".","6",".", ".", ".", ".", ".", ".", ".", "."],
30     [".","4",".", ".", ".", ".", ".", ".", ".", "."],
31     [".",".", ".", ".", ".", ".", ".", ".", ".", "."],
32     [".",".", ".", ".", ".", ".", ".", ".", ".", "."]]
33 solve_sudoku(board)
34 for row in board:
35     print(row)
```

Console I/O

```
Solved successfully.
[5,3,4,6,7,8,9,1,2]
[6,7,2,1,9,5,3,4,8]
[1,9,8,3,4,2,5,6,7]
[8,5,9,7,6,1,4,2,3]
[4,2,6,8,5,3,7,9,1]
[7,1,3,9,2,4,8,5,6]
[9,6,1,5,3,7,2,8,4]
[2,8,7,4,1,9,6,3,5]
[3,4,5,2,8,6,1,7,9]
```

5.  
Given nums and target, add '+' or '-' before every integer and return the number of expressions that evaluate to target.

Example 1:  
nums=[1,1,1,1,1], target=3  
Output: 5

Example 2:  
nums=[1], target=1  
Output: 1

</>OneCompiler Upgrade

+ AI Python Run

main.py

```
1 from functools import lru_cache
2
3 def target_sum(nums, target):
4     @lru_cache(None)
5     def dfs(i, total):
6         if i == len(nums):
7             return total == target
8         return dfs(i+1, total+nums[i]) + dfs(i+1, total-nums[i])
9     return dfs(0, 0)
10
11 print("Example 1:", target_sum([1,1,1,1,1], 3))
12 print("Example 2:", target_sum([1], 1))
```

Console I/O

Example 1: 5  
Example 2: 1

6.

Given an integer array arr, find the sum of min(b), where b ranges over every contiguous subarray of arr. Return the answer modulo  $10^9+7$ .

Example 1:

arr=[3,1,2,4]

Output: 17

Example 2:

arr=[11,81,94,43,3]

Output: 444

</>OneCompiler Upgrade

+ AI Python Run

main.py

```
1 def sum_subarray_mins(arr):
2     MOD = 10**9 + 7
3     stack = []
4     total = 0
5
6     for i, v in enumerate(arr + [0]):
7         while stack and arr[stack[-1]] > v:
8             j = stack.pop()
9             left = j - stack[-1] if stack else j + 1
10            right = i - j
11            total += arr[j] * left * right
12        stack.append(i)
13    return total % MOD
14
15 print("Example 1:", sum_subarray_mins([3,1,2,4]))
16 print("Example 2:", sum_subarray_mins([11,81,94,43,3]))
```

Console I/O

Example 1: 17  
Example 2: 444

7.

Given distinct candidates and a target, return all unique combinations whose sum equals target. The same number may be chosen unlimited times.

Example 1:

candidates=[2,3,6,7], target=7

Output: [[2,2,3],[7]]

Example 2:

candidates=[2,3,5], target=8

Output: [[2,2,2,2],[2,3,3],[3,5]]

</>OneCompiler Upgrade

+ AI Python Run

main.py

```
1 def combination_sum(candidates, target):
2     result = []
3     candidates.sort()
4
5     def backtrack(start, remain, path):
6         if remain == 0:
7             result.append(path.copy())
8             return
9         for i in range(start, len(candidates)):
10             if i >= remain: break
11             path.append(candidates[i])
12             backtrack(i+1, remain-candidates[i], path)
13             path.pop()
14
15     backtrack(0, target, [])
16     return result
17
18 print(combination_sum([2,3,6,7], 7))
19 print(combination_sum([2,3,5], 8))
```

Console I/O

Example 1: [[2, 2, 3], [7]]  
Example 2: [[2, 2, 2, 2], [2, 3, 3], [3, 5]]

8.

COMBINATION SUM 2: Given candidate numbers and a target, find all unique combinations whose sum equals target. Each number may be used only once and duplicate combinations are not allowed.

Example 1:

candidates=[10,1,2,7,6,1,5], target=8

Output: [[1,1,6],[1,2,5],[1,7],[2,6]]

Example 2:

candidates=[2,5,2,1,2], target=5

Output: [[1,2,2],[5]]

</>OneCompiler Upgrade

+ AI Python Run

main.py

```
1 def combination_sum2(candidates, target):
2     result = []
3     candidates.sort()
4
5     def backtrack(start, remain, path):
6         if remain == 0:
7             result.append(path.copy())
8             return
9         for i in range(start, len(candidates)):
10             if i >= remain: break
11             if i > 0 and candidates[i] == candidates[i-1]: continue
12             path.append(candidates[i])
13             backtrack(i+1, remain-candidates[i], path)
14             path.pop()
15
16     backtrack(0, target, [])
17     return result
18
19 print(combination_sum2([10,1,2,7,6,1,5], 8))
20 print(combination_sum2([2,5,2,1,2], 5))
```

Console I/O

Example 1: [[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]  
Example 2: [[1, 2, 2], [5]]

9.

Given an array nums of distinct integers, return all possible permutations.

Example 1:

nums=[1,2,3]

Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

Example 2:

nums=[0,1]

Output: [[0,1],[1,0]]

Example 3:

nums=[1]

Output: [[1]]

</OneCompiler Upgrade

+ AI Python Run

main.py

```
1 def permutations(nums):
2     result = []
3     used = [False] * len(nums)
4
5     def bt(path):
6         if len(path) == len(nums):
7             result.append(path.copy()); return
8         for i, n in enumerate(nums):
9             if not used[i]:
10                used[i] = True
11                path.append(n)
12                bt(path)
13                path.pop()
14                used[i] = False
15
16     bt([])
17     return result
18
19 print(permutations([1,2,3]))
20 print(permutations([0,1]))
21 print(permutations([1]))
```

Console I/O

Example 1: [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]  
Example 2: [[0, 1], [1, 0]]  
Example 3: [[1]]

10.

Given a collection nums that might contain duplicates, return all possible unique permutations.

Example 1:

nums=[1,1,2]

Output: [[1,1,2],[1,2,1],[2,1,1]]

Example 2:

nums=[1,2,3]

Output: all six permutations.

</OneCompiler Upgrade

+ AI Python Run

main.py

```
1 def unique_permutations(nums):
2     nums.sort()
3     result = []
4     used = [False] * len(nums)
5
6     def bt(path):
7         if len(path) == len(nums):
8             result.append(path.copy()); return
9         for i, n in enumerate(nums):
10            if i > 0 and nums[i] == nums[i-1] and not used[i-1]:
11                continue
12            used[i] = True; path.append(n)
13            bt(path)
14            path.pop(); used[i] = False
15
16     bt([])
17     return result
18
19 print(unique_permutations([1,1,2]))
20 print(unique_permutations([1,2,3]))
```

Console I/O

Example 1: [[1, 1, 2], [1, 2, 1], [2, 1, 1]]  
Example 2: [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]

11.

You and your friends are assigned the task of coloring a map with a limited number of colors. At each step you can choose an uncolored region and a valid color, followed by Alice and Bob. Maximize the number of regions you personally color. Also implement graph coloring for the undirected graph with edges=[(0,1),(1,2),(2,3),(3,0),(0,2)], n=4, using the minimum number of colors.

```
</>OneCompiler Upgrade + AI Python Run
```

```
main.py
1 def graph_coloring(edges, n):
2     adj = {set() for _ in range(n)}
3     for u, v in edges:
4         adj[u].add(v); adj[v].add(u)
5
6     color = [-1] * n
7     def bt(v, used):
8         if v == n:
9             return used
10        for c in range(used):
11            if all(color[u] != c for u in adj[v]):
12                color[v] = c
13                if bt(v+1, used): return used
14                color[v] = -1
15            color[v] = used
16            if bt(v+1, used+1): return used+1
17            color[v] = -1
18        return None
19    return bt(0, 0), color
20
21 edges = [(0,1), (1,2), (2,3), (3,0), (0,2)]
22 minimum, coloring = graph_coloring(edges, 4)
23 print("Minimum colors:", minimum)
24 print("One coloring:", coloring)
25 print("Maximum regions personally colored:", (4-2)//3)
```

```
Console I/O
Minimum colors: 3
One coloring: [0, 1, 2, 1]
Maximum regions personally colored: 2
```

12.

Using the same undirected graph edges=[(0,1),(1,2),(2,3),(3,0),(0,2)], n=4 and k=3, implement graph coloring with at most k colors and determine the maximum number of regions you personally color.

```
</>OneCompiler Upgrade + AI Python Run
```

```
main.py
1 def can_color_k(edges, n, k):
2     adj = {set() for _ in range(n)}
3     for u, v in edges:
4         adj[u].add(v); adj[v].add(u)
5     color = [1] * n
6
7     def bt(v):
8         if v == n: return True
9         for c in range(k):
10            if all(color[u] != c for u in adj[v]):
11                color[v] = c
12                if bt(v+1): return True
13                color[v] = 1
14        return False
15
16    ok, color = bt(0)
17    return ok, color
18
19 edges = [(0,1), (1,2), (2,3), (3,0), (0,2)]
20 ok, color = can_color_k(edges, 4, 3)
21 print("3-colorable:", ok)
22 print("Color assignment:", color)
23 print("Maximum regions personally colored:", (4-2)//3)
```

```
Console I/O
3-colorable: True
Color assignment: [0, 1, 2, 1]
Maximum regions personally colored: 2
```

13.

Given an undirected graph represented by edges and n vertices, determine whether a Hamiltonian cycle exists. A Hamiltonian cycle visits every vertex exactly once and returns to the starting vertex.

Example:

edges=[(0,1),(1,2),(2,3),(3,0),(0,2),(2,4),(4,0)], n=5.

</>OneCompiler

Upgrade

+ AI

Python

Run ▶

:

main.py

```
1 def hamiltonian_cycle(edges, n):
2     adj={set() for _ in range(n)}
3     for u,v in edges:
4         adj[u].add(v); adj[v].add(u)
5     path=[0]; used=[0]
6
7     def bt(u):
8         if len(path)==n:
9             return 0 in adj[u]
10            for v in adj[u]:
11                if v not in used:
12                    used.add(v); path.append(v)
13                    if bt(v): return True
14                    path.pop(); used.remove(v)
15            return False
16
17     ok=bt(0)
18     return ok, path
19
20 edges=[(0,1),(1,2),(2,3),(3,0),(0,2),(2,4),(4,0)]
21 print("Hamiltonian cycle exists:", hamiltonian_cycle(edges,5)[0])
```

Console I/O

Hamiltonian cycle exists: False

14.

Given an undirected graph represented by edges and n vertices, determine whether a Hamiltonian cycle exists.

Example:

edges=[(0,1),(1,2),(2,3),(3,0),(0,2)], n=4.

</>OneCompiler

Upgrade

+ AI

Python

Run ▶

:

main.py

```
1 def hamiltonian_cycle(edges, n):
2     adj={set() for _ in range(n)}
3     for u,v in edges:
4         adj[u].add(v); adj[v].add(u)
5     path=[0]; used=[0]
6
7     def bt(u):
8         if len(path)==n:
9             return 0 in adj[u]
10            for v in adj[u]:
11                if v not in used:
12                    used.add(v); path.append(v)
13                    if bt(v): return True
14                    path.pop(); used.remove(v)
15            return False
16
17     ok=bt(0)
18     return ok, path + ([0] if ok else [])
19
20 edges=[(0,1),(1,2),(2,3),(3,0),(0,2)]
21 ok, path=hamiltonian_cycle(edges,4)
22 print("Hamiltonian cycle exists:", ok)
23 print("Cycle:", path)
```

Console I/O

Hamiltonian cycle exists: True  
Cycle: [0, 1, 2, 3, 0]

15.

Generate all subsets of S containing n elements. Output each subset and discuss handling duplicate elements. For A=[1,2,3], the source lists: [], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3].

</>OneCompiler

Upgrade

+ AI

Python

Run ▶

:

main.py

```
1 def subsets(nums):
2     result=[]
3     def bt(i,path):
4         if i==len(nums):
5             result.append(path.copy()); return
6             bt(i+1,path)
7         path.append(nums[i]); bt(i+1,path); path.pop()
8     bt(0,[])
9     return sorted(result,key=lambda x: len(x),x))
10
11 A=[1,2,3]
12 for i in subsets(A):
13     print(i)
14 print("Duplicate elements: use a sorted list and skip equal choices at the same depth.")
```

Console I/O

```
[], [1], [2], [3], [1, 2], [1, 3], [2, 3], [1, 2, 3]
Duplicate elements: skip equal choices at the same recursion
depth.
```

16.

Given a set of unique integers and  $x=3$ , generate all subsets containing 3. For  $E=[2,3,4,5]$ , output the subsets containing 3. Also generate the power set for  $\text{nums}=[1,2,3]$  and  $\text{nums}=[0]$ .

</>OneCompiler

Upgrade

+ AI

Python

Run ▶

:

main.py

```
1 def subsets_containing(nums, x):
2     return [s for s in subsets(nums) if x in s]
3
4 def subsets(nums):
5     result=[]
6     def bt(i,path):
7         if i==len(nums):
8             result.append(path.copy()); return
9             bt(i+1,path)
10        path.append(nums[i]); bt(i+1,path); path.pop()
11    bt(0,[])
12    return sorted(result,key=lambda x: len(x),x))
13
14 E=[2,3,4,5]
15 print("Subsets containing 3:" subsets_containing(E,3))
16 print("Power set of [1,2,3]:" subsets([1,2,3]))
17 print("Power set of [0]:" subsets([0]))
```

Console I/O

```
Subsets containing 3: [[3], [2, 3], [3, 4], [3, 5], [2, 3, 4],
[2, 3, 5], [3, 4, 5], [2, 3, 4, 5]]
Power set of [1,2,3]: [[], [1], [2], [1,2], [3], [1,3], [2,3],
[1,2,3]]
Power set of [0]: [[], [0]]
```

17.

Given words1 and words2, a string b is a subset of string a if every letter in b occurs in a including multiplicity. Return all universal strings in words1.

Example 1:

words1=["amazon","apple","facebook","google","leetcode"], words2=["e","o"]

Output: ["facebook","google","leetcode"]

Example 2:

words2=["l","e"]

Output: ["apple","google","leetcode"]

main.py

```
1 from collections import Counter
2
3 def universal_words(words1, words2):
4     need=Counter()
5     for w in words2:
6         need[w]=Counter(w)
7
8     result=[]
9     for w in words1:
10         have=Counter(w)
11         if all(have[ch]>=count for ch,count in need.items()):
12             result.append(w)
13     return result
14
15 words1=["amazon","apple","facebook","google","leetcode"]
16 print(universal_words(words1,["e","o"]))
17 print(universal_words(words1,["l","e"]))
```

Console I/O

Example 1: ['facebook', 'google', 'leetcode']  
Example 2: ['apple', 'google', 'leetcode']